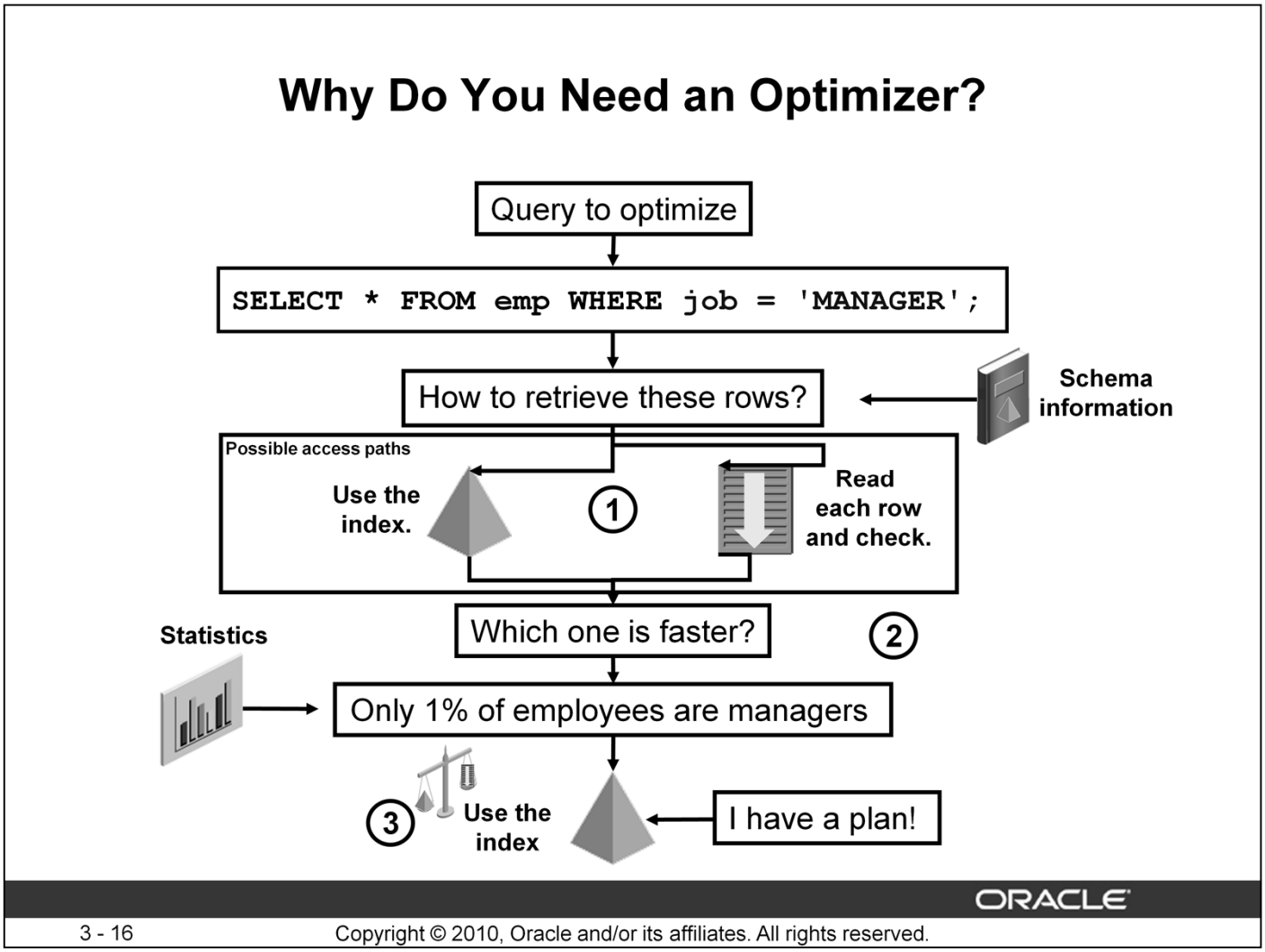


Why Do You Need an Optimizer?

The optimizer should always return the correct result as quickly as possible.

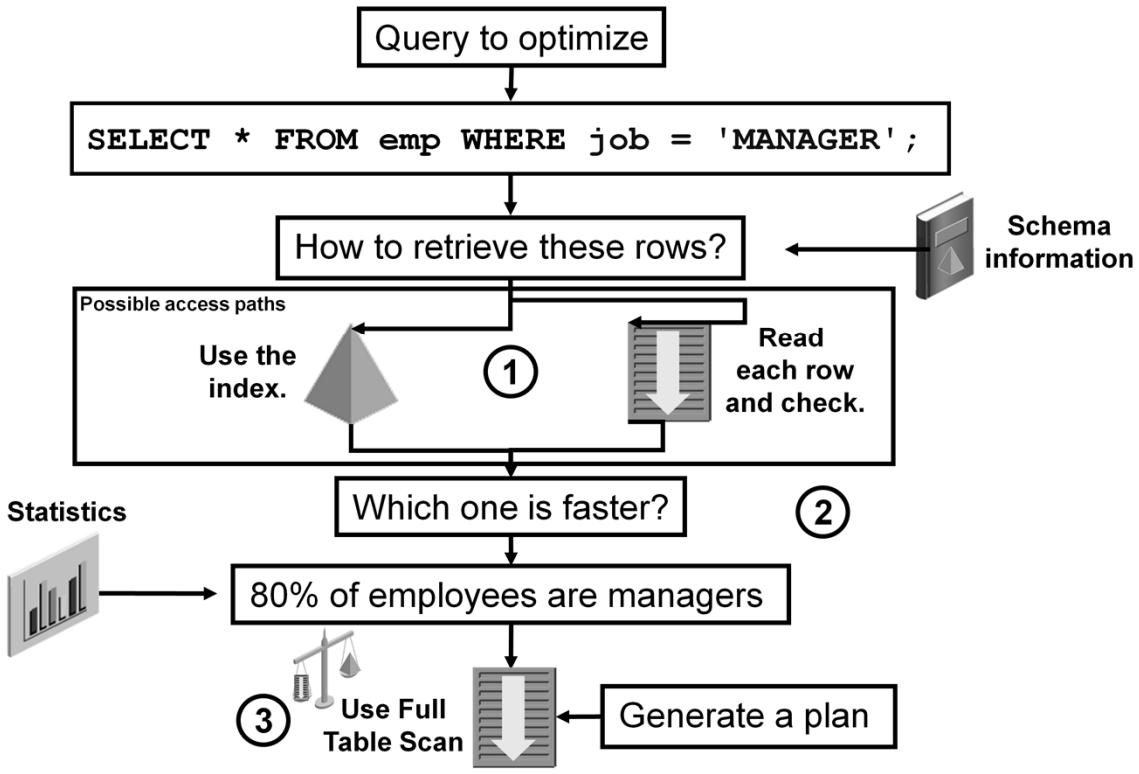
The query optimizer tries to determine which execution plan is most efficient by considering available access paths and by factoring in information based on statistics for the schema objects (tables or indexes) accessed by the SQL statement.

The query optimizer performs the following steps:

1. The optimizer generates a set of potential plans for the SQL statement based on available access paths.
2. The optimizer estimates the cost of each plan based on statistics in the data dictionary for the data distribution and storage characteristics of the tables, and indexes accessed by the statement.
3. The optimizer compares the costs of the plans and selects the one with the lowest cost.

Note: Because of the complexity of finding the best possible execution plan for a particular query, the optimizer's goal is to find a "good" plan that is generally called the best cost plan.

Why Do You Need an Optimizer?

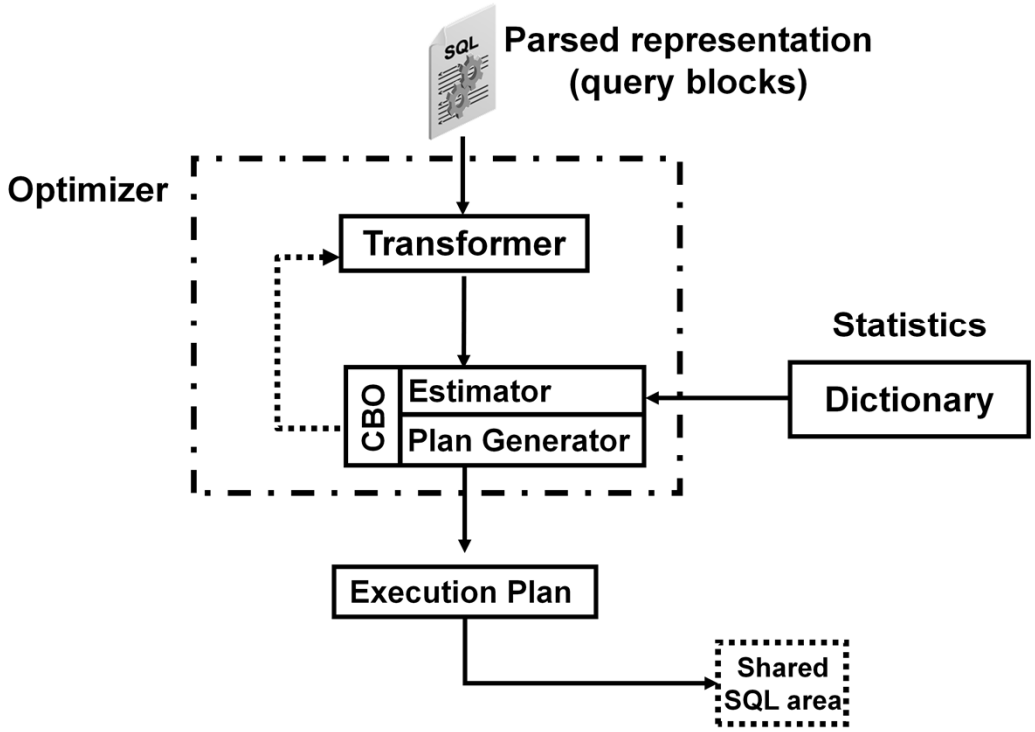


Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

Why Do You Need an Optimizer? (continued)

The example in the slide shows you that if statistics change, the optimizer adapts its execution plan. In this case, statistics show that 80 percent of the employees are managers. In the hypothetical case, a full table scan is probably a better solution than using the index.

Optimization During Hard Parse Operation



Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

Optimization During Hard Parse Operation

The optimizer creates the execution plan for a SQL statement.

SQL queries submitted to the system first run through the parser, which checks syntax and analyzes semantics. The result of this phase is called a parsed representation of the statement, and is constituted by a set of query blocks. A query block is a self-contained DML against a table. A query block can be a top-level DML or a subquery. This parsed representation is then sent to the optimizer, which handles three main functionalities: Transformation, estimation, and execution plan generation.

Before performing any cost calculation, the system may transform your statement into an equivalent
statement and calculate the cost of the equivalent statement. Depending on the version of Oracle Database, there are transformations that cannot be done, some that are always done, and some that are done, costed, and discarded.

The input to the query transformer is a parsed query, which is represented by a set of interrelated query blocks. The main objective of the query transformer is to determine if it is advantageous to change the structure of the query so that it enables generation of a better query plan. Several query transformation techniques are employed by the query transformer, such as transitivity, view merging, predicate pushing, subquery unnesting, query rewrite, star transformation, and `OR` expansion.

Transformer: OR Expansion Example

- Original query:

B*-tree Index

```
SELECT *
  FROM emp
  WHERE job = 'CLERK' OR deptno = 10;
```

- Equivalent transformed query:

```
SELECT *
  FROM emp
  WHERE job = 'CLERK'
UNION ALL
SELECT *
  FROM emp
  WHERE deptno = 10 AND job <> 'CLERK';
```

ORACLE

Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

Transformer: OR Expansion Example

If a query contains a WHERE clause with multiple conditions combined with OR operators, the optimizer transforms it into an equivalent compound query that uses the UNION ALL set operator, if this makes the query execute more efficiently.

For example, if each condition individually makes an index access path available, the optimizer can make the transformation. The optimizer selects an execution plan for the resulting statement that accesses the table multiple times using the different indexes and then puts the results together. This transformation is done if the cost estimation is better than the cost of the original statement.

In the example in the slide, it is assumed that there are indexes on both the JOB and DEPTNO columns. Then, the optimizer might transform the original query into the equivalent transformed query shown in the slide. When the cost-based optimizer (CBO) decides whether to make a transformation, the optimizer compares the cost of executing the original query using a full table scan with that of executing the resulting query.

Transformer: Subquery Unnesting Example

- Original query:

```
SELECT *
  FROM accounts
  WHERE custno IN
       (SELECT custno FROM customers);
```

- Equivalent transformed query:

```
SELECT accounts.*
  FROM accounts, customers
  WHERE accounts.custno = customers.custno;
```

Primary or unique key

ORACLE

Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

Transformer: Subquery Unnesting Example

To unnest a query, the optimizer may choose to transform the original query into an equivalent `JOIN` statement, and then optimize the `JOIN` statement.

The optimizer may do this transformation only if the resulting `JOIN` statement is guaranteed to return exactly the same rows as the original statement. This transformation allows the optimizer to take advantage of the join optimizer techniques.

In the example in the slide, if the `CUSTNO` column of the customers table is a primary key or has a `UNIQUE` constraint, the optimizer can transform the complex query into the shown `JOIN` statement that is guaranteed to return the same data.

If the optimizer cannot transform a complex statement into a `JOIN` statement, it selects execution plans for the parent statement and the subquery as though they were separate statements. The optimizer then executes the subquery and uses the rows returned to execute the parent query.

Note: Complex queries whose subqueries contain aggregate functions such as `AVG` cannot be transformed into `JOIN` statements.

Transformer: View Merging Example

- Original query:

Index

```
CREATE VIEW emp_10 AS
     SELECT empno, ename, job, sal, comm, deptno
     FROM emp
     WHERE deptno = 10;
```

```
SELECT empno FROM emp_10 WHERE empno > 7800;
```

- Equivalent transformed query:

```
SELECT empno
  FROM emp
  WHERE deptno = 10 AND empno > 7800;
```

ORACLE

3 - 21

Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

Transformer: View Merging Example

To merge the view's query into a referencing query block in the accessing statement, the optimizer replaces the name of the view with the names of its base tables in the query block and adds the condition of the view's query's `WHERE` clause to the accessing query block's `WHERE` clause.

This optimization applies to select-project-join views, which contain only selections, projections, and joins. That is, views that do not contain set operators, aggregate functions, `DISTINCT`, `GROUP BY`, `CONNECT BY`, and so on.

The view in this example is of all employees who work in department 10.

The query that follows the view's definition in the slide accesses the view. The query selects the IDs greater than 7800 of employees who work in department 10.

The optimizer may transform the query into the equivalent transformed query shown in the slide that accesses the view's base table.

If there are indexes on the `DEPTNO` or `EMPNO` columns, the resulting `WHERE` clause makes them available.

Transformer: Predicate Pushing Example

- Original query:

Index

```
CREATE VIEW two_emp_tables AS
SELECT empno, ename, job, sal, comm, deptno FROM emp1
 UNION
SELECT empno, ename, job, sal, comm, deptno FROM emp2;
```

```
SELECT ename FROM two_emp_tables WHERE deptno = 20;
```

- Equivalent transformed query:

```
SELECT ename
  FROM ( SELECT empno, ename, job,sal, comm, deptno
           FROM emp1 WHERE deptno = 20
          UNION
         SELECT empno, ename, job,sal, comm, deptno
           FROM emp2 WHERE deptno = 20 );
```

ORACLE

Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

Transformer: Predicate Pushing Example

The optimizer can transform a query block that accesses a nonmergeable view by pushing the query block's predicates inside the view's query.

In the example in the slide, the `two_emp_tables` view is the union of two employee tables. The view is defined with a compound query that uses the `UNION` set operator.

The query that follows the view's definition in the slide accesses the view. The query selects the IDs and names of all employees in either table who work in department 20.

Because the view is defined as a compound query, the optimizer cannot merge the view's query into the accessing query block. Instead, the optimizer can transform the accessing statement by pushing its predicate, the `WHERE` clause condition `deptno = 20`, into the view's compound query. The equivalent transformed query is shown in the slide.

If there is an index in the `DEPTNO` column of both tables, the resulting `WHERE` clauses make them available.

Transformer: Transitivity Example

- Original query:

Index

```
SELECT *
 FROM emp, dept
 WHERE emp.deptno = 20 AND emp.deptno = dept.deptno;
```

- Equivalent transformed query:

```
SELECT *
 FROM emp, dept
 WHERE emp.deptno = 20 AND emp.deptno = dept.deptno
   AND dept.deptno = 20;
```

ORACLE

Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

Transformer: Transitivity Example

If two conditions in the `WHERE` clause involve a common column, the optimizer sometimes can infer a third condition, using the transitivity principle. The optimizer can then use the inferred condition to optimize the statement.

The inferred condition can make available an index access path that was not made available by the original conditions.

This is demonstrated with the example in the slide. The `WHERE` clause of the original query contains two conditions, each of which uses the `EMP.DEPTNO` column. Using transitivity, the optimizer infers the following condition: `dept.deptno = 20`

If an index exists in the `DEPT.DEPTNO` column, this condition makes access paths available using that index.

Note: The optimizer only infers conditions that relate columns to constant expressions, rather than columns to other columns.

Cost-Based Optimizer

- Piece of code:
  - Estimator
  - Plan generator
- Estimator determines cost of optimization suggestions made by the plan generator:
  - Cost: Optimizer's best estimate of the number of standardized I/Os made to execute a particular statement optimization
- Plan generator:
  - Tries out different statement optimization techniques
  - Uses the estimator to cost each optimization suggestion
  - Chooses the best optimization suggestion based on cost
  - Generates an execution plan for best optimization

ORACLE

3 - 24 Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

Cost-Based Optimizer

The combination of the estimator and plan generator code is commonly called the cost-based optimizer (CBO).

The estimator generates three types of measures: selectivity, cardinality, and cost. These measures are related to each other. Cardinality is derived from selectivity and often the cost depends on cardinality. The end goal of the estimator is to estimate the overall cost of a given plan. If statistics are available, the estimator uses these to improve the degree of accuracy when computing the measures.

The main function of the plan generator is to try out different possible plans for a given query and pick the one that has the lowest cost. Many different plans are possible because of the various combinations of different access paths, join methods, and join orders that can be used to access and process data in different ways and produce the same result. The number of possible plans for a query block is proportional to the number of join items in the `FROM` clause. This number rises exponentially with the number of join items.

The optimizer uses various pieces of information to determine the best path: `WHERE` clause, statistics, initialization parameters, supplied hints, and schema information.

Estimator: Selectivity

$$\text{Selectivity} = \frac{\text{Number of rows satisfying a condition}}{\text{Total number of rows}}$$

- Selectivity is the estimated proportion of a row set retrieved by a particular predicate or combination of predicates.
- It is expressed as a value between 0.0 and 1.0:
  - High selectivity: Small proportion of rows
  - Low selectivity: Big proportion of rows
- Selectivity computation:
  - If no statistics: Use dynamic sampling
  - If no histograms: Assume even distribution of rows
- Statistic information:
  - `DBA_TABLES` and `DBA_TAB_STATISTICS` (`NUM_ROWS`)
  - `DBA_TAB_COL_STATISTICS` (`NUM_DISTINCT`, `DENSITY`, `HIGH/LOW_VALUE`,...)

ORACLE

3 - 25 Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

Estimator: Selectivity

Selectivity represents a fraction of rows from a row set. The row set can be a base table, a view, or the result of a join or a `GROUP BY` operator. The selectivity is tied to a query predicate, such as `last_name = 'Smith'`, or a combination of predicates, such as `last_name = 'Smith' AND job_type = 'Clerk'`. A predicate acts as a filter that filters a certain number of rows from a row set. Therefore, the selectivity of a predicate indicates the percentage of rows from a row set that passes the predicate test. Selectivity lies in a value range from 0.0 to 1.0. A selectivity of 0.0 means that no rows are selected from a row set, and a selectivity of 1.0 means that all rows are selected.

If no statistics are available, the optimizer either uses dynamic sampling or an internal default value, depending on the value of the `OPTIMIZER_DYNAMIC_SAMPLING` initialization parameter. When statistics are available, the estimator uses them to estimate selectivity. For example, for an equality predicate (`last_name = 'Smith'`), selectivity is set to the reciprocal of the number *n* of distinct values of `LAST_NAME` because the query selects rows that contain one out of *n* distinct values. Thus, even distribution is assumed. If a histogram is available in the `LAST_NAME` column, the estimator uses it instead of the number of distinct values. The histogram captures the distribution of different values in a column, so it yields better selectivity estimates.

Note: It is important to have histograms in columns that contain values with large variations in the number of duplicates (data skew).

Estimator: Cardinality

Cardinality = Selectivity * Total number of rows

- Expected number of rows retrieved by a particular operation in the execution plan
- Vital figure to determine join, filters, and sort costs
- Simple example:

```
SELECT days FROM courses WHERE dev_name = 'ANGEL';
```

  - The number of distinct values in `DEV_NAME` is 203.
  - The number of rows in `COURSES` (original cardinality) is 1018.
  - Selectivity = 1/203 = 4.926*e-03
  - Cardinality = (1/203)*1018 = 5.01 (rounded off to 6)

ORACLE

Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

Estimator: Cardinality

The cardinality of a particular operation in the execution plan of a query represents the estimated number of rows retrieved by that particular operation. Most of the time, the row source can be a base table, a view, or the result of a join or `GROUP BY` operator.

When costing a join operation, it is important to know the cardinality of the driving row source. With nested loops join, for example, the driving row source defines how often the system probes the inner row source.

Because sort costs are dependent on the size and number of rows to be sorted, cardinality figures are also vital for sort costing.

In the example in the slide, based on assumed statistics, the optimizer knows that there are 203 different values in the `DEV_NAME` column, and that the total number of rows in the `COURSES` table is 1018. Based on this assumption, the optimizer deduces that the selectivity of the `DEV_NAME='ANGEL'` predicate is 1/203 (assuming there are no histograms), and also deduces the cardinality of the query to be (1/203)*1018. This number is then rounded off to the nearest integer, 6.

Estimator: Cost

- Cost is the optimizer's best estimate of the number of standardized I/Os it takes to execute a particular statement.
- Cost unit is a standardized single block random read:
  - 1 cost unit = 1 SRds
- The cost formula combines three different costs units into standard cost units.

$$\text{Cost} = \frac{\boxed{\text{Single block I/O cost}\ \texttt{\#SRds*sreadtim}} + \boxed{\text{Multiblock I/O cost}\ \texttt{\#MRds*mreadtim}} + \boxed{\text{CPU cost}\ \texttt{\#CPUCycles/cpuspeed}}}{\texttt{sreadtim}}$$

`#SRds`: Number of single block reads
`#MRds`: Number of multiblock reads
`#CPUCycles`: Number of CPU Cycles

`Sreadtim`: Single block read time
`Mreadtim`: Multiblock read time
`Cpuspeed`: Millions instructions per second

Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

Estimator: Cost

The cost of a statement represents the optimizer's best estimate of the number of standardized inputs/outputs (I/Os) it takes to execute that statement. Basically, the cost is a normalized value in terms of a number of single block random reads

The standard cost metric measured by the optimizer is in terms of number of single block random reads, so one cost unit corresponds to one single block random read. The formula shown in the slide combines three different cost units:

- Estimated time to do all the single-block random reads
- Estimated time to do all the multiblock reads
- Estimated time for the CPU to process the statement into one standard cost unit

The model includes CPU costing because in most cases CPU utilization is as important as I/O; often it is the only contribution to the cost (in cases of in-memory sort, hash, predicate evaluation, and cached I/O).

This model is straightforward for serial execution. For parallel execution, necessary adjustments are made while computing estimates for `#SRds`, `#MRds`, and `#CPUCycles`.

Note: `#CPUCycles` includes CPU cost of query processing (pure CPU cost) and CPU cost of data retrieval (CPU cost of the buffer cache get).

Plan Generator

```
select e.last_name, d.department_name
from   employees e, departments d
where  e.department_id = d.department_id;
```

```
Join order[1]:  DEPARTMENTS[D]#0  EMPLOYEES[E]#1
 NL Join:  Cost: 41.13  Resp: 41.13  Degree: 1
 SM cost: 8.01
 HA cost: 6.51
Best:: JoinMethod: Hash
Cost: 6.51  Degree: 1  Resp: 6.51  Card: 106.00
Join order[2]:  EMPLOYEES[E]#1  DEPARTMENTS[D]#0
 NL Join:  Cost: 121.24  Resp: 121.24  Degree: 1
 SM cost: 8.01
 HA cost: 6.51
Join order aborted
Final cost for query block SEL$1 (#0)
All Rows Plan:
Best join order: 1
+-----------------------------------------------------------------+
| Id  | Operation           | Name        | Rows  | Bytes | Cost  |
+-----------------------------------------------------------------+
| 0   | SELECT STATEMENT    |             |       |       |     7 |
| 1   |  HASH JOIN          |             |   106 |  6042 |     7 |
| 2   |   TABLE ACCESS FULL | DEPARTMENTS|    27 |   810 |     3 |
| 3   |   TABLE ACCESS FULL | EMPLOYEES   |   107 |  2889 |     3 |
+-----------------------------------------------------------------+
```


Copyright © 2010, Oracle and/or its affiliates. All rights reserved.

Plan Generator

The plan generator explores various plans for a query block by trying out different access paths, join methods, and join orders. Ultimately, the plan generator delivers the best execution plan for your statement. The slide shows you an extract of an optimizer trace file generated for the select statement. As you can see from the trace, the plan generator has six possibilities, or six different plans to test: Two join orders, and for each, three different join methods. It is assumed that there are no indexes in this example.

To retrieve the rows, you can start to join the `DEPARTMENTS` table to the `EMPLOYEES` table. For that particular join order, you can use three possible join mechanisms that the optimizer knows: Nested Loop, Sort Merge, or Hash Join. For each possibility, you have the cost of the corresponding plan. The best plan is the one shown at the end of the trace.

The plan generator uses an internal cutoff to reduce the number of plans it tries when finding the one with the lowest cost. The cutoff is based on the cost of the current best plan. If the current best cost is large, the plan generator tries harder (in other words, explores more alternate plans) to find a better plan with lower cost. If the current best cost is small, the plan generator ends the search swiftly because further cost improvement is not significant. The cutoff works well if the plan generator starts with an initial join order that produces a plan with a cost close to optimal. Finding a good initial join order is a difficult problem.

Note: Access path, join methods, and plan are discussed in more detail in the lessons titled “Optimizer Operators” and “Interpreting Execution Plans.”